STA232B Project 3

Eric Kalosa-Kenyon

10 March 2018

Abstract

In this report, I examine mouse mortality data and the beta-binomial model presented in Brook et al. (1997) to more accurately model the observations.

Contents

Paper summary **1**
- Background . . . 2
- Purpose . . . 2
- Data . . . 2
- Methods . . . 3
- Results . . . 3

Monte-Carlo Expectation Maximization (MCEM) **3**
- Expectation maximization (EM) . . . 3
- Monte-Carlo (MC) . . . 4
- Metropolis-Hastings (MH) . . . 4
- Putting it all together . . . 4

Analysis **5**
- Outline of the MCEM for mouse litter mortality . . . 5
- Results . . . 6

Discussion **8**
- Standard errors . . . 8

Sources **8**

Appendix **8**
- Random effects (α) . . . 9
- Program output . . . 11
- Source Code . . . 12

Paper summary

In this section, I sumarize the background, purpose, data, methods, and results of the paper "Finite Mixture Models for Proportions" by Brooks, Morgan, Ridout, and Pack published in Biometrics (53, 1097-1115) in September of 1997. In brief, the researchers implement a mixture model alternative for mouse mortality data. The frequency of normal outliers in the mouse litter mortality data rendered a normal model less useful - the mixture of beta-binomial and binomial models allowed for infrequent, but not 6σ infrequent, full litter mortality events undeniably present in the data.

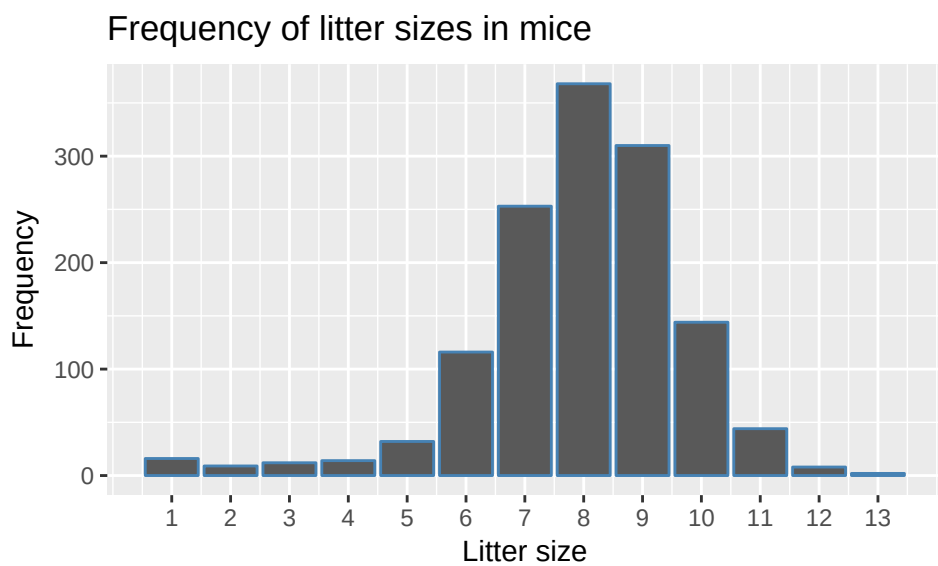
Background

The only certianty in life is death - yet we know surprisingly little about the phenomena. In daring to write about such a poingant topic, we statisticians may relegate ourselves to the quantification of phenomenological rates, likelihoods, and the like - we will eschew metaphysics and its relations to keep our scope limited and thereby tractable. In this spirit, Brook et al. examined mouse litter mortality data - that is, they improved the model used to describe the number of deaths in litters.

Purpose

Brook et al. (1997) improved the modeling of mouse mortality data by introducing a mixture of beta-binomial and binomial model components.

Data



The above plot depicts a summary of the data by showing the frequencies of each litter size. Missing from the above plot is the rates of death in each litter size. The following table displays some of the raw data where $Y_{ij} = 1$ if the jth implants in the ith litter died and $Y_{ij} = 0$ if it survived.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
y	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
litter	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	17	18	18

Note that the first 16 litters have only one implant, of which only the 16th died. That is, $Y_{i,1} = 0$ for $i \in 1:15$ and $Y_{16,1} = 1$. The litter labeled $i = 17$ had two implants in it, both of which survived, so $Y_{17,j} = 0$ for $j \in 1:2$.

The total number of litters is $N = 1328$.

Methods

The beta-binomial model, reproduced form the paper, is

$$P(X = x \mid n) = \binom{n}{p} \prod_{r=0}^{x-1} (\mu + r\theta) \frac{\prod_{r=0}^{n-x-1}(1 - \mu + r\theta)}{\prod_{r=0}^{n-1}(1 + r\theta)} = Q(x, n)$$

where n is the number of "implants" or fetuses in a litter, and x is the number that died.

The mixture model, reproduced again from the paper, is

$$P(X = x \mid n) = \gamma Q(x, n) + (1 - \gamma) \binom{n}{x} \nu^x (1 - \nu)^{n-x}$$

where ν is the probability of death.

In the paper, the authors fit the following model:

$$P(X = x \mid n) = Q(x, n) \left(1 + \frac{\phi}{2} g(x, n, \mu, \theta)\right)$$

for a function $g(\cdot)$ specified in Paul (1985).

Results

Table 11 on p1110 of Brook et al. describes shows which models best fit which datasets using AIC/BIC. The table is not reproduced here for brevity.

Monte-Carlo Expectation Maximization (MCEM)

In this section, I provide a high level summary of the Monte-Carlo expectation maximization (MCEM) method targeted at practitioners.

Expectation maximization (EM)

To understand MCEM, it is necessary to first understand expectation maximization (EM) alone. In EM, there are two steps: calculating the Q function from the expected value and the maximization of the Q function with respect to the parameter of interest. More explicitly, a pseudocode rendition of the EM algorithm follows:

```
data y
parameter u
using f(y|u)
initialize guess for u -> u0
while not converged:
    Q(u|u_t) = E_{u_t}[l(u|y)] # E step
    u_{t+1} = argmax_u(Q(u|u_t))
    if delta(u_t, u_{t+1}) < epsilon:
        converged = True
```

Monte-Carlo (MC)

The other half of MCEM is the Monte-Carlo methodology. In brief, this method uses the fact that the emperical distribution F_n will converge to the true distribution F once enough samples have been taken. An easy example is the calculation of pi, the pseudocode for which follows:

```
dist(x,y) = sqrt(x^2+y^2)
samples_in = 0
for i in 1:N do
    u1, u2 = sample(uniform(0,1), twice)
    if dist(u1,u2) < 1
        samples_in += 1
pi = 4*(samples_in/N)
```

Metropolis-Hastings (MH)

The Metropolis-Hastings (MH) algorithm is an MC method useful for for generating a decent approximiation of the true distribution when the analytic form is unavailable. It creates a chain of samples from the distribution that are used as an emperical distribution approximiation by sampling randomly at uniform candidate values from the support, comparing their likelihoods, and accepting the candidate with a probability $\alpha = min(\frac{f(x_{t+1})}{f(x_t)}, 1)$. Basically, we always move uphill if we can, otherwise we jump with a likelihood proportional to how bad the jump would be.

The pseudocode for MH follows:

```
xs = []
xs[1] = initial guess
for i in 1:N do
    x = sample(unif(support))
    xt = xs[i]
    a = f(x)/f(xt)
    if a > 1:
        xs[i+1] = x
    else:
        u = sample(unif(0,1))
        if u < a:
            xs[i+1] = x
        else:
            xs[i+1] = xt
return(xs)
```

Putting it all together

MCEM is useful when you want to get maximum likelihood estimates for parameters and unobserved data in a situation where sampling from the likelihood function is difficult. To put it to use, one performs MH to sample the unobserved data from the likelihood function and extracts the MLE for those data. Then, one calculates the Q function and extracts the MLEs for the unknown parameters, repeating both steps until the algorithm converges.

Analysis

In this section, I expose the analytical techniques used to obtain estimates of the unknown parameters in the logistic model for mouse litter mortality with a random litter effect.

It's useful to note that the y_is are conditionally independent given α_i. That is, I can optimize each α_i independently. Furthermore, it's useful to notice that

$$E[l(y, \alpha \mid \mu, \sigma)] = E[l(y \mid \alpha, \mu)] + E[l(\alpha \mid \sigma)]$$

That is, I can maximize over μ and σ separately.

The known data are the mouse litter mortality observations y_{ij}. The unobserved data are the litter random effects α_i. The unknown parameters are μ and σ in

$$\alpha_i \sim N(0, \sigma^2)$$

and

$$logit(P(y_{ij} = 1 \mid \alpha)) = \mu + \alpha_i$$

Outline of the MCEM for mouse litter mortality

In this section I provide a brief plan of the algorithm used to recover the unknown parameters μ and σ along with the unobserved random litter effects α_i.

The pseudocode for my algorithm follows:

```
data y
unknown mu, sigma
unobserved alpha

converged = F
while (not converged) do
    alpha = e_step(y, mu, sigma)
    mu, sigma = m_step(y, alpha)
    if (delta(alpha, mu, sigma) < epsilon) then do
        converged = T

define function e_step(y, m, s) as
    for a_i in alphas do
        samples = []
        for j in 1:(number MH samples) do
            samples.append(sample.metropolishastings(l(y_i, m, s, a)))
        a_i = most common sample
    return(alphas)

define function m_step(y, alphas) as
    u = argmax(f(y, u | alpha))
    s = argmax(f(s | alphas))
    return(u, s)
```

Furthermore, it's helpful to notice that the random effects in the expectation step can be sampled from an easily computable conditional distribution:

$$f(\alpha_i \mid y_i) = \frac{1}{N_{y_i}} f(y_i \mid \alpha_i) f(\alpha) \tag{1}$$

$$f(\alpha) = N(0, \sigma^2) \tag{2}$$

$$logit(f(y_i \mid \alpha_i)) = \mu + \alpha_i \tag{3}$$

The normalization factor (N_y) due to the marginal distribution ($f(y)$) is unavailable, necessitating a Metropolis-Hastings aproach for sampling, as implemented in the following section.

Results

Running the MCEM algorithm as described above with the following parameters

$$\epsilon = 0.01, \quad \text{max.iterations} = 30, \quad \text{range} = [-3, 3]$$

yields $\mu = -2.1$ and $\sigma = 0.1563403$ after 30.651 seconds.

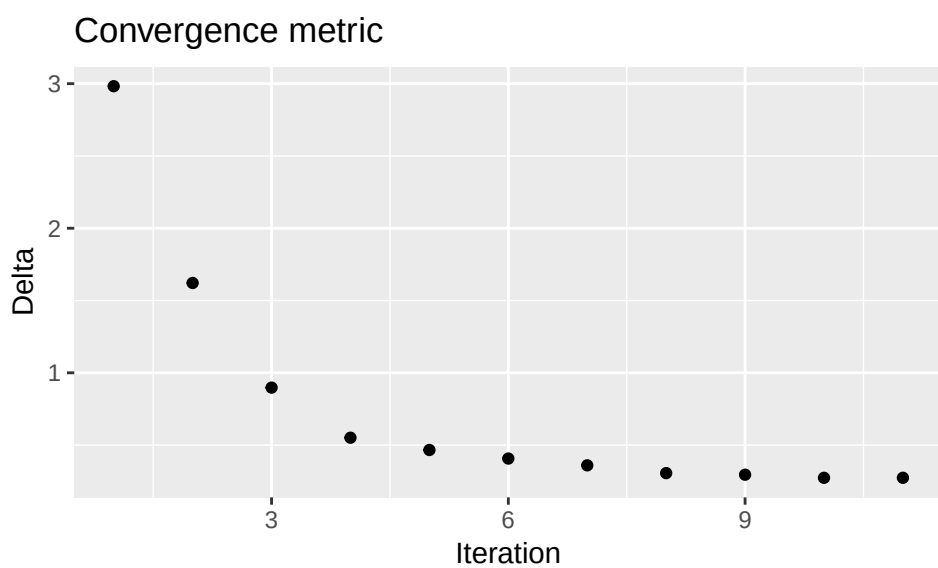
The log-likelihood of the MCEM fit model is -3001.151824, and, for comparison, the log-likelihood of the GLMER fit model is -4110.3392515. For all that work, I at least improved the fit a little. That is, the likelihood ratio for the two models is

$$\lambda = \frac{f(mcem)}{f(glmer)} = e^{l(mcem) - l(glmer)} = \infty \tag{4}$$

$$\log(\lambda) = l(mcem) - l(glmer) = 1109.1874275 \tag{5}$$

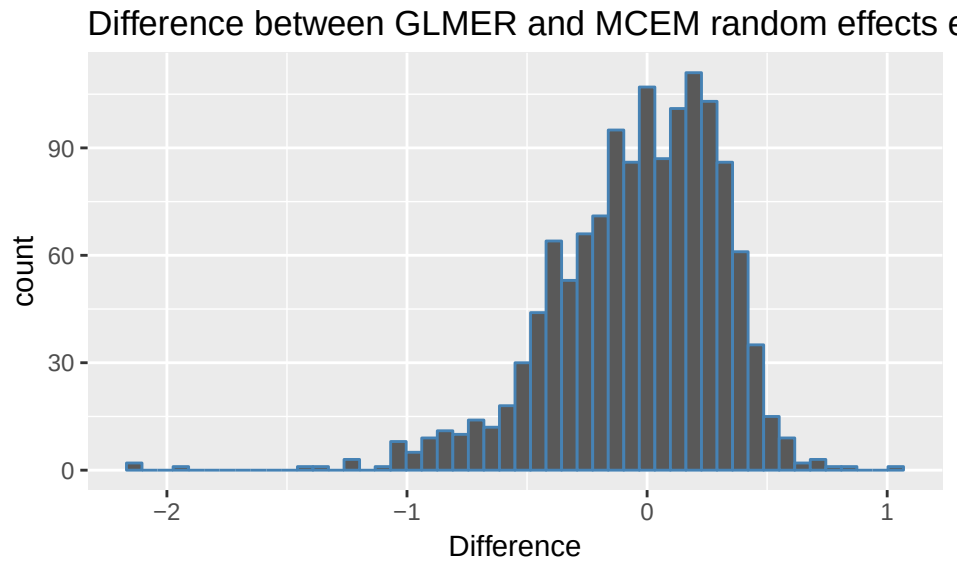
Rate of convergence

The rate of convergence is depicted by the following figure:

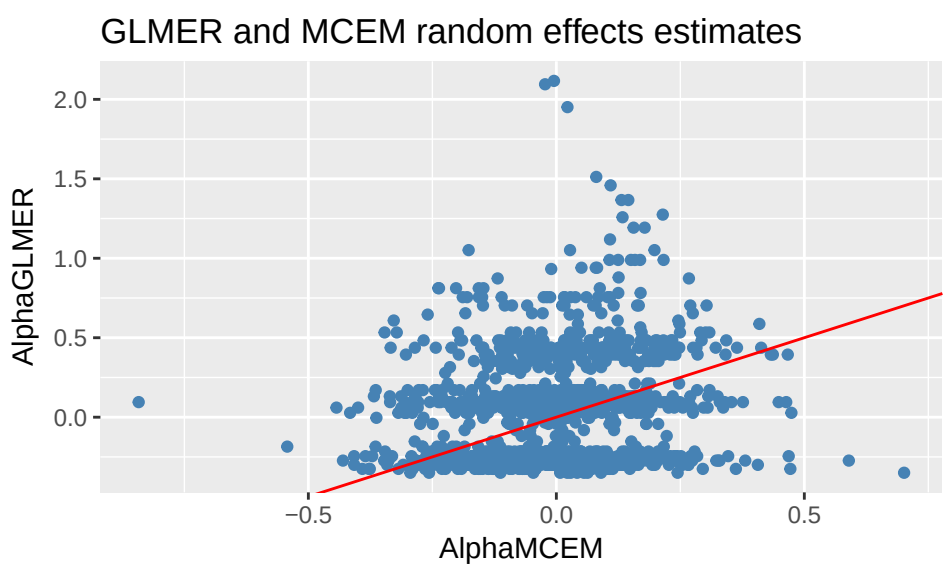


Random effects (α)

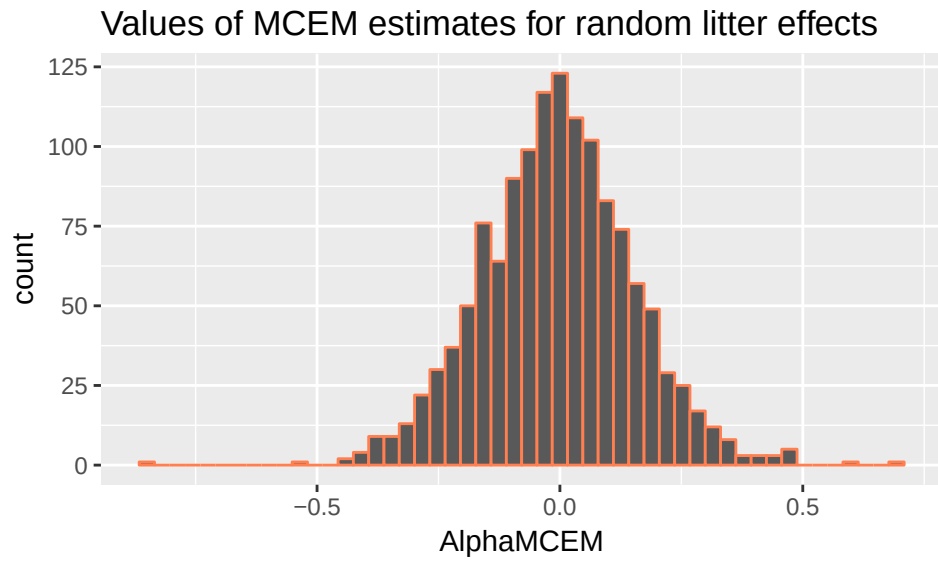
The difference between the MCEM and GLMER estimates of the random effects are depicted in the following figure:



The following plot compares the values of the GLMER and MCEM estimates for the random effects. If they were in complete agreement, all the blue dots would lie on the red line.



The following plot depicts the estimates for random litter effects.



The full list of random effects are included in the appendix.

Discussion

The methods used in this project are highly versatile and well suited to a broad diversity of problems. Here, however, it may have been overkill to use the Metropolis-Hastings algorithm to sample from $f(\alpha_i \mid y_i)$. As a 1-dimensional problem, a simple grid search may have worked faster and more accurately. Regardless, MCEM works well here and produces a model with better fit than the naive glmer() function.

Standard errors

In future work, one might be interested in estimating the standard errors of the random effects. To do so in the MCEM setup, it might be useful to bootstrap the sequential estimates for individual random effects after a burn-in period once the distribution over the random effects has stabilized. More explicitly, one might resample $\alpha_i^{(t)}$ for all $\tau < t < T$ where τ is the iteration past which the burn-in period has ended and T is the last iterated sample from the target distribution. Then, the sampled $\{\alpha_i^{(t)}\}_{\tau<t<T}^{(b)}$ are used to calculate summary statistics like standard error.

Sources

1. Brooks et al. (1997). Finite mixture models for proportions. Biometrics 53, 1097-1115.
2. Paul (1985). A three-parameter generalisation of the binomial distribution. Communications in Statistics Theory and Methods 14(6), 1497-1506.

Appendix

In this section I present the data and source code used to generate this report.

Random effects (α)

-0.0779799	-0.1702101	-0.1046636	-0.2021760	0.2454687	-0.0235612	-0.0871947	-0.0417408	-0.0140860
0.1072858	0.0546735	0.2432593	-0.2365774	-0.0929620	0.2257218	-0.2261945	-0.0845492	-0.0705199
0.1811462	0.0716978	-0.0348139	0.0876094	-0.0511363	0.0894226	-0.0593228	0.1371071	0.0577417
-0.1751745	-0.0346361	-0.0538728	-0.1564058	0.0263337	0.0365226	-0.0449271	-0.1793787	-0.1753609
0.1812469	-0.0035202	-0.1561746	-0.2379081	-0.1154079	0.2406346	-0.1345136	0.0061797	-0.0365386
-0.0953674	0.0100710	0.2512823	-0.1470312	0.1497921	-0.1475380	0.0493861	-0.0295203	0.1512187
0.1122700	0.0430517	-0.2921330	0.1082801	-0.0952800	0.0089552	-0.2518331	0.0216858	0.1804077
-0.2746417	0.0509849	-0.0184488	0.0224378	0.1316496	0.2825341	0.0489641	0.2738011	0.0399430
0.0000805	0.0126771	-0.0970785	0.3266284	0.0183753	0.0056227	-0.1087347	0.2970550	-0.0992374
-0.0473152	0.0456178	-0.0690040	0.2250923	0.1383267	-0.2430977	-0.2081990	0.1266013	-0.0905830
0.2053445	0.1806104	0.0801629	-0.1682932	-0.1623412	0.0904921	-0.0686827	-0.0029904	-0.0411038
0.0083922	-0.0451984	0.0105215	-0.1667098	-0.0903935	0.0940470	-0.1793065	-0.0681435	0.0598146
-0.2492884	-0.2063146	0.0926559	-0.0002495	0.3290460	0.3213580	0.2269361	0.0824162	-0.0675683
-0.0924917	-0.2304369	0.0744539	-0.1465612	0.0059266	-0.0787910	-0.0322965	-0.1820382	0.0408750
-0.0159273	0.3088904	-0.0415433	0.0687175	0.2739543	0.1340002	0.0186753	0.1490148	-0.3061188
0.0223143	0.0350292	-0.0327452	-0.2183039	-0.3383470	0.0601336	-0.0541941	0.2263207	-0.1446067
-0.0818258	0.0102100	0.0658798	-0.1567686	-0.1240980	0.1981651	-0.0947092	0.0556192	0.0663124
-0.1848003	-0.0147311	0.0860219	0.1404427	-0.0859707	-0.0661438	0.0087905	0.0631498	0.1624382
0.0556026	0.0623402	0.0130139	0.0714664	-0.1550818	0.0047474	0.0443483	0.0420581	0.1857233
0.1162659	0.2838302	-0.0127134	-0.2194265	-0.0315080	0.0919832	-0.2384971	-0.0158504	-0.0088868
-0.0528297	-0.2447712	0.0130169	0.0979431	-0.1390434	-0.0236792	-0.1124073	-0.0347736	0.2754027
0.0121388	-0.0536236	-0.1174607	0.0509701	0.0223018	0.0074283	0.1150047	0.0593922	0.1475207
-0.0209999	-0.3641477	0.0553629	-0.1898220	-0.2315970	0.2192088	-0.1601662	-0.0830597	-0.4157964
0.1248738	0.0916824	-0.1769938	-0.1247736	0.1437213	-0.0362582	-0.1139211	-0.0348631	0.0239947
0.1698935	0.0349304	-0.0478909	-0.1379006	0.0954999	0.0421102	-0.0581437	0.0834973	-0.1411733
-0.0010508	-0.2998205	0.0567066	-0.1977797	0.0167418	0.1388492	-0.0532318	-0.1967014	-0.0560104
0.0262112	-0.2411569	-0.0245583	0.0069659	-0.0455546	0.2465324	-0.1077138	0.0445391	0.1706004
-0.2269341	-0.2691921	0.1340256	0.1300577	0.2047902	0.2259608	0.1056031	-0.0540798	-0.1821841
0.2225521	0.0903793	0.2000591	0.0978203	-0.1622154	0.2281284	0.1294556	-0.0678627	0.0359758
-0.0699312	-0.1947391	-0.3129384	-0.1161645	-0.2120172	0.1152478	0.1007906	0.1857851	-0.3186106
-0.0519030	0.3010237	-0.1700204	0.0544000	-0.1188025	-0.2113086	-0.1466835	0.4325299	-0.0953745
-0.0125386	0.0476752	-0.0864958	0.1107794	-0.0028292	0.3642601	0.0192722	-0.1366693	0.0915659
-0.0716959	0.1277305	-0.3357495	0.0864629	-0.1623410	0.1680887	0.0258328	0.1270327	-0.1441346
-0.0884170	0.0858724	0.1384233	-0.2078679	0.1108361	-0.1076257	-0.1274951	-0.0318972	0.4739597
-0.0696201	-0.3462959	-0.0334514	0.1940611	-0.0427843	-0.0429072	0.2543946	0.0810537	0.0040784
-0.1093862	0.3085297	-0.0422232	0.0109836	-0.1475999	-0.2852321	-0.1355804	-0.0109267	-0.1499044
0.0695321	-0.1981746	0.1764879	-0.1098783	0.0514911	0.0952218	0.1330154	0.3411648	-0.0009761
-0.2855049	0.1719730	-0.0063982	-0.1772529	-0.0265321	0.0087840	0.0439553	-0.1048434	0.0703204
0.1512022	-0.0917492	-0.0080987	-0.0062564	0.0525910	-0.0290125	-0.3411401	-0.0327363	0.0360358
-0.0390411	-0.0296717	-0.2363219	-0.0937142	0.0370320	-0.0109775	0.4062950	-0.0980915	-0.2904542
0.0369673	0.0277991	0.0398497	-0.3471225	-0.1923004	-0.1138037	0.1449554	0.0758508	0.0013430
-0.1181875	-0.3217537	0.0578610	0.0214479	-0.0566198	-0.1512820	0.0050654	0.2401338	-0.3022050
0.0125732	0.1056037	-0.0154616	0.0547689	-0.2377503	-0.3340850	-0.1493862	0.0998615	-0.1376841
-0.1300167	0.2901909	0.1466763	0.1949654	-0.0529412	-0.0200884	-0.0121020	0.1981504	-0.0446311
0.0087238	0.2492385	0.1012657	-0.0553812	-0.0218288	0.1652682	-0.2058585	0.1274892	0.0446775
-0.1467775	-0.0548962	0.3252292	-0.0193435	0.0483280	0.0942028	0.0290766	0.1194523	-0.0466051
0.0269922	-0.0811367	-0.1433299	-0.0041274	-0.0424206	-0.0107228	-0.1438638	0.4666170	0.0776529
0.0433649	-0.1182717	0.3063305	0.5897337	0.2490812	-0.0857215	-0.1750446	-0.0151485	0.1810327
-0.2593253	0.2673052	0.0779345	-0.1921946	0.0557450	0.0814764	-0.1609415	-0.0172737	-0.2041462
0.1451116	0.1557161	0.1806905	-0.1993043	0.0153969	0.0305383	-0.0173624	0.1483093	0.0718566
0.1313641	0.1781404	0.0178059	-0.0542760	-0.2210736	0.4129768	0.0148889	-0.1088236	0.0722913

-0.2092426	-0.0410902	-0.1288277	0.0412607	0.0525454	0.2477300	-0.2473879	0.0376778	-0.1184401
0.2166640	0.1427981	0.1727570	-0.1679809	-0.1379094	0.0101670	-0.0155402	0.4365291	-0.1661519
-0.2699529	-0.0871386	-0.1051413	-0.2886771	0.1268528	0.2757695	-0.0319583	0.0139187	0.0206302
-0.0051403	-0.0182567	-0.0834738	0.0003585	-0.0996789	0.1736030	0.1051746	0.2847744	0.1385062
-0.1850541	0.2411677	-0.1005422	0.1749326	-0.1514761	-0.1476962	0.0716030	0.1967039	0.0995314
-0.3645175	-0.0824543	0.0351495	-0.0185346	0.0648359	-0.0933589	-0.1051228	-0.1441928	0.0048037
-0.5425192	0.0427956	-0.0191612	-0.2701994	-0.0015081	0.1132166	0.1137935	0.0919463	0.2006833
-0.1133076	0.1272975	-0.1054093	-0.2305057	0.1625705	0.0637870	0.0179134	0.2296677	-0.0536683
-0.2402609	-0.0141232	-0.0218884	0.0527991	0.0448197	0.0508575	-0.2788799	0.1119270	0.1782309
-0.2696844	0.0831968	0.0188793	-0.0397476	-0.8421044	0.0167512	0.1415500	0.0163768	-0.0913902
-0.0558670	0.1235855	0.1821502	0.1589052	0.0563631	0.0842646	0.1329144	0.0059496	0.1866064
0.1601897	-0.4077485	0.2702480	0.1375203	-0.2101056	0.1082714	-0.0950888	0.1870663	0.1542630
-0.1575393	-0.0203053	0.1581013	0.1484052	-0.1788661	-0.1799236	-0.0059428	0.0274138	0.0542911
-0.1240467	0.1655748	-0.2308801	0.0350947	0.0595818	0.0383996	-0.1085852	-0.0509520	0.0575151
-0.2313921	-0.2143660	-0.2839617	0.0716091	0.3393237	0.0285198	0.0162848	0.2361964	-0.0668932
-0.2017615	-0.2623091	-0.0388571	0.1123912	-0.0044188	-0.1886486	-0.0164588	0.2174728	-0.1293232
0.1582961	0.0936123	0.0219180	-0.2821073	0.0038318	-0.0253769	-0.1458558	-0.0081526	0.2060206
-0.0223867	-0.1705505	-0.0724601	0.2346667	0.1907913	0.1034581	0.0532650	0.0498444	-0.0308189
0.0683104	0.0405084	-0.0396517	0.1132859	0.4483262	0.0154632	-0.0758868	-0.3032928	0.0517170
-0.1536406	0.0175866	-0.1074099	-0.2828098	-0.0042642	0.0605850	0.1205722	0.1057743	0.1763982
0.0198182	0.0786101	-0.1360327	-0.0467568	-0.0514547	-0.0124243	-0.1927450	0.1055587	-0.0396481
0.1887998	-0.1843975	0.1738011	0.0440480	0.0397314	-0.1103420	-0.0760050	-0.1020840	-0.0243444
-0.2189873	-0.1357313	-0.0659712	-0.0283605	-0.2900803	-0.1495970	0.0180811	0.0118298	0.1572993
0.1831091	-0.0620422	0.1266541	-0.1963199	-0.3049979	-0.0189083	-0.1987742	-0.1479089	0.0107576
0.2475493	0.1711297	-0.0104089	0.0556731	0.0401382	-0.1562615	-0.0598238	-0.0588370	-0.1426595
0.4093695	-0.1466008	0.1880449	0.0951420	-0.2550228	-0.1767856	0.0775700	0.1628691	0.0091143
0.0434146	-0.3311945	-0.0263806	-0.0480853	-0.0289497	0.0276466	0.1918594	0.0793501	-0.0989553
0.2481963	0.2298525	0.0928094	-0.2440812	-0.1531979	0.1979282	-0.0037159	0.2701255	0.0833340
0.0794991	-0.1850389	0.0742118	-0.0741875	-0.2208465	-0.0227212	0.0139801	0.1156199	-0.1349517
0.0821900	-0.0141972	0.0353611	-0.0852348	0.1766404	0.1023666	-0.3064597	0.3030358	-0.1267194
0.0506688	-0.2353697	-0.0219801	-0.0213890	0.3296026	-0.1003750	0.0784613	0.0746082	0.1960167
0.2146461	-0.0877911	-0.0206960	0.0614546	-0.1510190	-0.1285412	-0.1463522	0.1662121	-0.0629473
-0.2236041	-0.0597699	-0.0523600	-0.0925920	0.1211007	0.0223749	-0.1474601	0.0165826	-0.1833305
0.1331974	-0.2025335	0.1886755	0.1045357	-0.0141670	-0.1489370	0.0029660	-0.0893833	-0.0275485
-0.1263392	-0.0472933	-0.0448693	0.0452524	-0.1863989	-0.1958577	0.2377085	-0.1040185	-0.0489958
-0.0329851	-0.0498965	0.1571141	-0.0533478	0.1201178	-0.0130148	0.0990804	0.1246673	0.0907469
0.0501272	0.2709083	-0.1222826	-0.1075813	-0.0352909	0.0340486	-0.1614930	0.1071293	0.2751607
-0.0203936	0.0196556	-0.0768873	-0.2470529	0.4635006	0.0070719	0.1828276	0.1689346	-0.0102835
0.0363887	-0.0535289	-0.0134124	-0.0083329	0.1831900	-0.3086395	-0.2115695	0.2162663	-0.0047787
0.0814853	0.1611302	0.2346664	0.0235957	-0.0812719	-0.0407951	-0.0589903	0.1503799	-0.2944024
0.1858879	-0.0012785	-0.3678536	-0.0310131	-0.1369183	-0.2265290	-0.1944261	0.1585661	-0.0464774
-0.1381245	-0.1141672	0.1539562	-0.0619699	0.0833711	0.1308225	-0.0772031	0.1335152	0.0077141
0.0947160	-0.2557158	0.1399914	0.1019127	0.0603691	-0.4075401	-0.1105307	0.0804890	0.0629657
-0.0453096	-0.0013970	0.0445449	-0.1637615	-0.3352254	0.0360396	0.2549795	-0.3183457	-0.1126670
-0.0159653	-0.0433678	0.0900718	0.1235036	-0.0351860	-0.1411735	0.1880052	0.0854383	0.7011789
-0.0159517	-0.2464772	0.1195862	0.1111222	0.1528179	-0.0135481	0.0730453	-0.1986359	0.2445578
-0.0607820	0.1678737	0.1670195	0.3225737	0.0515108	0.1264603	-0.4433254	0.0318907	0.0418640
0.0073414	-0.0901986	0.1534460	0.0550113	-0.0445332	0.0364663	-0.0904498	-0.1567412	0.0900919
0.0485125	-0.1683171	0.0375954	0.1158902	0.1151937	-0.0072527	-0.1575840	-0.1641866	0.0288162
-0.2034079	-0.0505991	-0.1243409	-0.0674638	0.3531690	-0.2267982	-0.0220446	-0.1277563	0.0220064
0.1558345	-0.1686826	0.1304149	-0.0715352	0.3350473	0.0074535	-0.0640618	0.0496126	0.0557504
-0.1887996	0.1946518	-0.0030256	-0.0481892	-0.1537144	0.0774102	-0.1463991	-0.3911535	-0.2566621
0.1384638	-0.0332428	-0.0259539	-0.1383772	-0.1915702	-0.2888547	-0.1358591	-0.1913978	-0.0931031
0.0258758	0.2225843	0.1074183	-0.0485127	-0.0185677	-0.0087406	-0.3122800	0.3619745	0.1103627

-0.3453242	-0.0041718	-0.0438652	-0.4302105	0.0790020	0.0277694	-0.1739853	-0.0571863	-0.3627883
-0.0538106	0.4685105	0.2336744	0.0361034	0.0318064	-0.1953508	0.0887265	0.1136175	0.0665147
0.0234054	0.1805777	-0.1610349	0.1381741	0.0124994	0.1479991	0.0385144	0.0093874	0.0229071
-0.1429613	-0.3848716	-0.0996491	-0.1944951	0.0729985	0.0360745	-0.0106197	0.0323440	0.0109773
-0.0047914	-0.0443797	0.0170474	-0.1621689	0.2243551	-0.1026697	-0.0351860	-0.0417774	-0.0333442
-0.0986207	0.0848683	0.2028419	-0.1551982	-0.0692747	-0.0928210	-0.0965805	0.2953422	-0.0445587
0.2787503	0.0038276	0.0736164	-0.0007504	-0.2784549	0.1989859	0.3049702	0.1482060	0.1136677
0.0778070	-0.1687136	0.2314945	-0.1363069	-0.1672005	-0.1120791	-0.0399329	0.0096824	0.0162390
-0.0749460	0.0590470	0.1805845	-0.2520819	0.0879718	-0.0715529	-0.2070781	0.0253027	-0.1470471
0.0834343	-0.1227881	0.0156964	0.0458327	-0.0922678	-0.1398652	-0.1993600	0.1037761	-0.0018763
0.0675951	-0.0359287	0.2001142	0.0623336	-0.0615651	0.0110182	-0.0298388	0.0167694	0.0214683
-0.2068456	-0.0678819	0.1159877	-0.0686312	0.0088262	-0.0919345	0.0115931	-0.1556540	0.0633256
-0.2522474	-0.0944514	0.1228052	-0.0851264	-0.0571740	0.0053779	0.1192313	0.0867133	-0.2678206
0.2050285	-0.1457600	-0.0628869	-0.1755296	-0.0660223	-0.0073738	0.1897774	-0.0644314	-0.0792149
-0.1534484	0.0054187	0.0409857	0.1603451	0.2178433	0.1090803	0.1702601	-0.0739602	-0.1380469
0.0052723	0.2372345	-0.1934685	-0.1002464	-0.0586884	-0.0880464	0.0689426	-0.0890537	0.0903744
-0.0708088	-0.0049602	-0.0353634	0.0048718	-0.2334368	-0.2571961	-0.0658781	-0.2236177	-0.0355814
0.1690916	0.3464580	-0.2675658	0.1209984	-0.2167370	0.0083598	0.0587757	0.0726143	0.1645585
-0.0196347	-0.3712209	0.1099308	0.0132408	0.1054475	0.0320484	0.0744593	-0.1368630	-0.0626761
0.0430361	-0.0765191	0.1419476	0.0777605	0.0768720	-0.1084552	0.0952468	-0.1235622	0.0244738
-0.0458787	-0.1616968	0.1299066	-0.0194446	0.2100817	-0.1018057	-0.1269929	-0.0680024	0.0065021
-0.0583769	0.0242956	0.1702653	0.2381652	-0.0630517	-0.1754177	-0.0423958	0.0658258	0.0627122
-0.0841763	0.0894227	-0.0497727	-0.0850548	-0.1495890	0.0583725	-0.0591745	-0.0434820	0.0614552
-0.2059366	-0.2227220	-0.1899620	0.1834372	-0.2111157	-0.1623550	-0.1073517	0.0233138	-0.0954875
0.0490928	0.0003273	0.1070180	-0.0894863	0.0690674	-0.1533782	-0.1388310	0.1073945	-0.2144330
-0.0938787	-0.0213189	0.0219932	-0.1477974	0.1079560	-0.2593927	-0.0829701	0.0069568	0.1236294
-0.0709512	-0.1072581	-0.1118404	0.0160187	0.0610249	0.0492855	0.0467273	-0.0756018	-0.3278106
-0.0604319	-0.0984441	0.2158859	0.1649470	0.0293124	0.1398571	0.0512564	-0.2897185	0.2460058
0.1598552	-0.0040847	0.1458925	-0.0156483	-0.0464988	-0.0209222	-0.1635315	-0.2647236	0.1255782
0.1016029	-0.2448862	0.2929378	-0.1416222	0.1004805	-0.1484252	-0.1087728	-0.2306016	-0.0231137
-0.1269882	0.0384381	-0.0511042	-0.1348430	-0.1443013	-0.1072780	0.0183584	-0.0608727	0.1186244
-0.0706816	-0.0546959	0.3426506	-0.0143851	-0.0171471	0.2511548	-0.3997218	-0.1526150	-0.0989843
-0.0330709	0.0077090	-0.0901101	0.0544697	-0.1114877	-0.1161598	-0.0429249	-0.0940885	0.0387955
-0.0215300	0.0061476	0.0697009	-0.0272792	0.0675655	0.0561894	0.1275772	-0.0687400	-0.0728209
-0.0102732	-0.2541577	0.1237453	0.3808909	0.0809844	0.2659244	-0.2795007	-0.0414642	0.0175918
-0.2775084	-0.2539669	0.1368672	-0.1105227	-0.1025789	0.1934269	0.0719133	-0.1274386	-0.2240109
-0.0963346	0.2145934	0.1819617	0.0524434	-0.0212609	0.1653208	0.3363703	-0.3761619	0.1693125
-0.0323579	0.1900176	0.0245326	-0.0133515	0.2674596	0.2663486	0.0901004	0.4718367	-0.1223245
-0.1435911	0.1080378	0.0670664	-0.1790659	0.0767321	-0.0754055	-0.1542891	-0.0524494	0.1094353
0.1174203	-0.0455063	-0.0594518	0.1408189	-0.0159752	0.0994884	-0.0093760	-0.1484632	NA
-0.0672977	0.2839611	-0.1169561	-0.0357339	0.1100778	0.1585787	-0.1083938	0.0647934	NA
0.1274262	-0.0190109	0.1181279	-0.0220016	0.0154917	-0.2606949	0.0360512	-0.1370813	NA
-0.0761684	0.1518708	-0.0850981	0.2513379	0.3762241	0.0059354	0.0108694	-0.3777246	NA

Program output

```
[1] "loop 0"
[1] "alpha mse 1.75463773385602"
[1] "u -1"
[1] "s 0.637934009910178"
[1] "delta 2.98268530014902"
[1] "loop 1"
[1] "alpha mse 0.293835498011355"
```

```
[1] "u -1.85"
[1] "s 0.449877581911466"
[1] "delta 1.6215417355299"
[1] "loop 2"
[1] "alpha mse 0.134774943959391"
[1] "u -2.1"
[1] "s 0.349901489779348"
[1] "delta 0.897695837588981"
[1] "loop 3"
[1] "alpha mse 0.118978250451298"
[1] "u -2.1"
[1] "s 0.291276494703581"
[1] "delta 0.551083435351029"
[1] "loop 4"
[1] "alpha mse 0.130399945089645"
[1] "u -2.1"
[1] "s 0.238040152087478"
[1] "delta 0.466131936013698"
[1] "loop 5"
[1] "alpha mse 0.125521673449298"
[1] "u -2.1"
[1] "s 0.212257668531744"
[1] "delta 0.407005425263196"
[1] "loop 6"
[1] "alpha mse 0.121229234852789"
[1] "u -2.1"
[1] "s 0.189364793134914"
[1] "delta 0.359833713853732"
[1] "loop 7"
[1] "alpha mse 0.125127937187878"
[1] "u -2.1"
[1] "s 0.169845066460247"
[1] "delta 0.306147572491318"
[1] "loop 8"
[1] "alpha mse 0.127518123086702"
[1] "u -2.1"
[1] "s 0.155987795336138"
[1] "delta 0.295230548735708"
[1] "loop 9"
[1] "alpha mse 0.12569394774214"
[1] "u -2.1"
[1] "s 0.156340303145683"
[1] "delta 0.273717374028638"
[1] "loop 10"
[1] "alpha mse 0.126313922441186"
[1] "u -2.1"
[1] "s 0.155350537353604"
[1] "delta 0.273795876977965"
[1] "failing to converge"
```

Source Code

```
# Author: Eric Kalosa-Kenyon, Chi Po Choi
```

```
## @knitr setup
rm(list=ls())

library(lme4)
library(ggplot2)

# Load data
nlist <- list()
nlist[[1]] <- c(15,1)
nlist[[2]] <- c(6,1,2)
nlist[[3]] <- c(6,6)
nlist[[4]] <- c(7,2,3,NA,2)
nlist[[5]] <- c(16,9,3,3,1)
nlist[[6]] <- c(57,38,17,2,2)
nlist[[7]] <- c(119,81,45,6,1,NA,NA,1)
nlist[[8]] <- c(173,118,57,16,3,NA,NA,NA,1)
nlist[[9]] <- c(136,103,50,13,6,1,1)
nlist[[10]] <- c(54,51,32,5,1,NA,NA,NA,NA,1)
nlist[[11]] <- c(13,15,12,3,1)
nlist[[12]] <- c(NA,4,3,1)
nlist[[13]] <- c(NA,NA,1,NA,NA,NA,NA,1)
df <- vector()
count <- 0
for (k in 1:13) {
for (l in 1:length(nlist[[k]])) {
    if(!is.na(nlist[[k]][l])){
    for(m in 1:nlist[[k]][l]){
        count <- count + 1
        df <- rbind(df, cbind( c(rep(0,k-l+1),rep(1,l-1) ) , count))
    }}
}}
df <- data.frame(y = df[,1], litter = factor(df[,2]))

initial <- glmer(y ~ 1 + (1|litter), family = binomial, data = df)

## @knitr littersize
# Count litter size frequency
litter.sizes = c()
for(i in levels(df$litter)){
    litsz = length(which(df$litter == i))
    litter.sizes = c(litter.sizes, litsz)
}
litter.sizes.df = data.frame(Size=unique(litter.sizes))
litter.size.frqs = c()
for(i in unique(litter.sizes)){
    frq = length(which(litter.sizes == i))
    litter.size.frqs = c(litter.size.frqs, frq)
}
litter.sizes.df$Freqency = litter.size.frqs

p.litsz = ggplot(data.frame(X=litter.sizes), aes(x=X)) +
    geom_bar(color="steelblue") +
    labs(
```

```
         title="Frequency of litter sizes in mice",
         x="Litter size", y="Frequency"
         ) +
    scale_x_continuous(breaks=litter.sizes.df$Size)

## @knitr mcem

N = length(levels(df$litter))
M = 10^2 # number of times to sample from the l() for MH
alps = rep(1, N) # initial guess for random effects
u = 0 # initial guesses
s = 1

eps = 0.01 # convergence criteria
max.it = 30
ran = c(-3, 3) # range for alphas, mu, sig (start, length)

lygiva = function(ys, a, u){
    # log-likelihood function

    r = 0
    p = exp(u+a)/(1+exp(u+a))
    mp = 1/(1+exp(u+a))
    for(y in ys){
        if(y == 1){
            r1 = p
        }else{
            r1 = mp
        }
        r = r+log(r1)
    }
    return(r)
}

lagivy = function(a, ys, u, s){
    # non-normalized posterior on alpha
    # f(a|y) = 1/Ny*f(y|a)f(a)
    r1 = lygiva(ys, a, u)
    r2 = log(dnorm(a, mean=0, sd=s))
    r = r1+r2
    return(r)
}

lugivya = function(u, alps){
    stopifnot(length(alps) == N)
    r = 0
    for(i in 1:N){
        ixs = which(df$litter == i)
        ys = df$y[ixs]
        a = alps[i]
        r = r + lygiva(ys, a, u)
    }
    return(r)
}
```

```
mh = function(f, ...){
    # metropolis hastings sample and maximization
    samp = matrix(NA, M, 1)
    us = runif(M, min=ran[1], max=ran[2])
    checks = runif(M-1)
    samp[1] = us[1]

    for(i in 2:M){
        x = samp[i-1]
        xt = us[i]
        a1 = exp(f(xt, ...)-f(x, ...)) # f is a log-likelihood

        # sometimes the x and/or xt are terrible guesses resulting in an
        # overflow for the log likelihood i.e. f(x,...) = -Inf or ditto for xt
        # In that case, pick the one that's not -Inf, if they're _both_ shite,
        # pick a maximizer of f(x,...)
        if(is.na(a1 > checks[i-1]) || is.null(a1 > checks[i-1])){
            aa = seq(from=ran[1], to=ran[2], by=5*eps)
            samp[i] = aa[which.max(f(aa,...))]
            # little bit of a hack but it keeps the train chugging
        }else{
            if(a1 > checks[i-1]){
                samp[i] = xt
            }else{
                samp[i] = x
            }
        }
    }

    # return the most common sample from the metrop-hastings algorithm
    tab = table(samp)
    ix = which.max(tab)[1]
    r = as.numeric(names(tab[ix]))
    return(r)
}

e_step = function(u, s){
    # find the best unobserved alphas given the data and parameters
    alps = matrix(NA, N, 1)

    for(a in 1:N){
        # if(a %% 100 == 0){print(paste("in e step", a))}

        # sample the alphas from the likelihood function
        ixs = which(df$litter == a)
        ys = df$y[ixs]
        alps[a] = mh(lagivy, ys=ys, u=u, s=s)
    }

    print(paste("alpha mse", 1/N*sum((ranef(initial)[[1]]-alps)^2)))
    return(alps)
}
```

```
m_step = function(alps){
    # find maximum likelihood estimates for mu and sigma
    s = sd(alps)
    u0 = mean(alps)
    us = seq(from=ran[1], to=ran[2], by=eps*5)
    out = lugivya(us, alps=alps)
    u = us[which.max(out)[1]]

    print(paste("u", u))
    print(paste("s", s))
    return(c(u, s))
}

delta = function(a,x,y, a1,x1,y1){
    r = max(abs(quantile(a-a1, probs=c(0.1, 0.9))))
    r = max(r, abs(x-x1), abs(y-y1))
    print(paste("delta", r))
    return(r)
}

i = 0
conv = F
ds = c()
stime = proc.time()

{
sink("./analysis.out")
while(1){

    print(paste("loop", i))

    alps1 = e_step(u, s) # takes a while... hashtag thanksForLoops
    p = m_step(alps1)
    u1 = p[1]
    s1 = p[2]

    # convergence criteria
    d = delta(alps,u,s, alps1,u1,s1)
    ds = c(ds, d)
    if(d < eps){
        conv = T
        print("wow it actually converged")
        break
    }else if(i>max.it){
        print("taking too long")
        break
    }else if(length(ds) > 1){
        if(abs(ds[i+1]-ds[i]) < eps/10){
            print("failing to converge")
            break
        }
    }

    # update parameters
```

```
    i = i+1
    alps = alps1
    u = u1
    s = s1
}
etime = proc.time()
sink()
}

# plots
p.alpha.df = alps - ranef(initial)[[1]]
colnames(p.alpha.df) = c("Difference")
p.alpha.df$AlphaMCEM = alps
p.alpha.df$AlphaGLMER = ranef(initial)[[1]][,1] # list containing dataframe

p.alpha1 = ggplot(p.alpha.df, aes(x=Difference)) +
    geom_histogram(bins=50, color="steelblue") +
    labs(title="Difference between GLMER and MCEM random effects estimates")

p.alpha2 = ggplot(p.alpha.df, aes(x=AlphaMCEM, y=AlphaGLMER)) +
    geom_point(color="steelblue") +
    labs(title="GLMER and MCEM random effects estimates") +
    geom_abline(color="red")

p.alpha3 = ggplot(p.alpha.df, aes(x=AlphaMCEM)) +
    geom_histogram(bins=50, color="coral") +
    labs(title="Values of MCEM estimates for random litter effects")

p.delta.df = data.frame(Iteration=1:length(ds), Delta=ds)
p.delta = ggplot(p.delta.df, aes(x=Iteration, y=Delta)) +
    geom_point() +
    labs(title="Convergence metric")

# calculate the log likelihoods of the two models
lmcem = 0
lglmer = 0
glmer_alps = ranef(initial)[[1]]
glmer_u = fixef(initial)[1]
glmer_s = as.data.frame(VarCorr(initial))$sdcor
for(i in 1:N){
    yi = df$y[which(df$litter == i)]
    lmcem = lmcem + lagivy(alps[i], yi, u, s)
    lglmer = lglmer + lagivy(glmer_alps[i,1], yi, glmer_u, glmer_s)

}
```